INDIVIDUAL MODULE PROJECT REPORT: MODULE 3

NEON POSTGRESQL VECTOR STORE & SPRING SECURITY ARCHITECTURE

Project Based Learning (PBL) — Java Projects (5th Semester)
Department of Computer Science & Engineering
Arya College of Engineering & I.T., Jaipur

MODULE PROFILE

- **Module Name**: Module 3: Neon PostgreSQL Vector Store (`pgvector`) & Spring Security Architecture
- **Student Name**: **Vaibhav Tank** (Database Architect & Security Engineer)
- **University Roll No**: 22EAICS062
- **Dedicated Git Branch**: `feature/member3-database-security`
- **Project Guide / Mentor**: Er. Ram Babu Buri (Assistant Professor, Dept. of CSE)
- **Core Technologies**: Neon Serverless PostgreSQL 16, `pgvector` 0.7+, Spring Security 6, JJWT 0.12, Bucket4j, Kaggle Corpus Vector Store

---

1. MODULE SCOPE & OBJECTIVES

1.1 Architectural Boundary

Module 3 forms the persistence, indexing, and security foundation of DocuQuery AI. It provisions and configures the cloud-native serverless PostgreSQL instance on Neon, maintains relational schemas, optimizes the `pgvector` HNSW index for high-dimensional cosine similarity searches across the Kaggle Academic Research Paper Corpus, and implements end-to-end security via Spring Security 6, JWT, and Bucket4j rate limiting.

1.2 Core Objectives

1. **Cloud Database Provisioning**: Provision serverless PostgreSQL 16 on Neon with PgBouncer connection pooling and TLS 1.3 encryption.
2. `pgvector` **Extension & Schema Architecture**: Design relational tables with cascading constraints and high-dimensional vector columns (`vector(1536)`).
3. **HNSW Cosine Distance Tuning**: Tune HNSW graph construction parameters ($m=16, ef_construction=64$) for sub-100ms vector lookups.
4. **Stateless Authentication**: Implement Spring Security 6 filter chains with HMAC-SHA256 signed JWT tokens and BCrypt password encryption.
5. **Role-Based Access Control (RBAC)**: Enforce granular security boundaries (`ROLE_ADMIN` , `ROLE_RESEARCHER` , `ROLE_STUDENT`).
6. **API Rate Limiting & Auditing**: Guard REST endpoints using Bucket4j token bucket filters (30 req/min per IP) and log queries to the database.

2. DETAILED MODULE ARCHITECTURE & DDL



3. KEY IMPLEMENTATION CLASSES & SCHEMAS

3.1 PostgreSQL DDL with `pgvector` HNSW Index

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(100) UNIQUE NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    filename VARCHAR(255) NOT NULL,
    content_type VARCHAR(100),
    file_size BIGINT,
    total_pages INT,
    total_chunks INT,
    status VARCHAR(50) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE document_chunks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,
    page_number INT,
    content TEXT NOT NULL,
    token_count INT,
    embedding vector(1536),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- HNSW Vector Index for Sub-100ms Cosine Distance Lookup
CREATE INDEX idx_document_chunks_embedding_hnsw
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

3.2 `DocumentChunkRepository.java` (Native Cosine Search)

```
package com.pbl.rag.repository;

import com.pbl.rag.model.DocumentChunk;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.UUID;

@Repository
public interface DocumentChunkRepository extends JpaRepository<DocumentChunk, UUID> {

    @Query(value = """
        SELECT dc.*, 1 - (dc.embedding <=> cast(:queryVector as vector)) as similarity
        FROM document_chunks dc
        WHERE dc.embedding IS NOT NULL
        ORDER BY dc.embedding <=> cast(:queryVector as vector) ASC
        LIMIT :topK
        """, nativeQuery = true)
    List<DocumentChunk> findTopKSimilarChunks(
        @Param("queryVector") String queryVector,
        @Param("topK") int topK
    );
}
```

3.3 Security Configuration (`application.yml`)

```
spring:
  datasource:
    url: jdbc:postgresql://${NEON_HOST:ep-bold-glade.us-east-2.aws.neon.tech}/${NEON_DB:docuquery}?
sslmode=require
    username: ${NEON_USER:neondb_owner}
    password: ${NEON_PASS:npg_secret}
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2
      idle-timeout: 30000
      connection-timeout: 20000
app:
  jwt:
    secret: 404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970
    expiration-ms: 86400000 # 24 hours
```

---

4. MATHEMATICAL FORMULATION & HNSW ANALYSIS

4.1 Cosine Distance Operator `<=>` in `pgvector`

For unit vectors $\mathbf{u}$ and $\mathbf{v}$, the cosine distance is: $$D_{\text{cos}}(\mathbf{u}, \mathbf{v}) = 1 - (\mathbf{u} \cdot \mathbf{v}) = 1 - \sum_{i=1}^{1536} u_i v_i$$

4.2 HNSW Tuning Trade-Offs

- **Parameter $m = 16$**: Defines the maximum number of bidirectional connections per element in the HNSW graph. Higher m increases recall at the cost of memory.
- **Parameter $ef_construction = 64$**: Defines the search depth during index construction. Setting $ef = 64$ provides an optimal balance between build time and sub-100ms lookup speed.

---

5. UNIT TESTING & SECURITY VERIFICATION

```
@SpringBootTest
class SecurityAndDatabaseTest {

    @Autowired
    private DocumentChunkRepository chunkRepository;

    @Test
    void testPgVectorNativeCosineSearch_ExecutesUnder150ms() {
        long start = System.currentTimeMillis();
        String zeroVector = "[" + "0.01,".repeat(1535) + "0.01]";
        List<DocumentChunk> results = chunkRepository.findTopKSimilarChunks(zeroVector, 4);
        long latency = System.currentTimeMillis() - start;

        assertNotNull(results);
        assertTrue(latency < 150, "Vector search latency exceeded 150ms");
    }
}
```

---

6. GITHUB COMMITS & WEEKLY TIMELINE SUMMARY

- **Branch**: `feature/member3-database-security`
- **Author**: **Vaibhav Tank**
- **Commit 1**: `feat(database): configure Neon PostgreSQL schema and pgvector HNSW indexing`
- **Commit 2**: `feat(security): implement Spring Security 6 stateless JWT filter chain`
- **Commit 3**: `feat(security): add Role-Based Access Control (RBAC) and Bucket4j rate limiting`
- **Verification**: Merged cleanly into `develop` and `main` with 100% test pass rate.